Lab 7: Inheritance and OOP Problem-Solving

Exercise 1: Student Management System

Concepts: Inheritance, constructors, `super()`, private fields, getters, arrays, searching

Given: `StudentTest.java`

```
public class StudentTest {
    public static void main(String[] args) {
        Student[] students = {
            new Student("Alice", 20, "S101"),
            new Student("Bob", 18, "S102"),
            new Student("Carol", 23, "S103"),
            new Student("David", 21, "S104")
        };

        Student oldest = findOldest(students);

        System.out.println("Oldest Student: " + oldest.getName());
        System.out.println("Age: " + oldest.getAge());
        System.out.println("Student ID: " + oldest.getStudentId());
    }

    public static Student findOldest(Student[] students) {
        // TODO: Find and return the oldest student.
        return null;
    }
}
```

Your tasks

Create the following classes:

1. `Person.java`

- Private fields: `name` and `age`
- Public constructor: `Person(String name, int age)`
- Public getter methods: `getName()` and `getAge()`

2. `Student.java`

- Extend `Person`.
- Private field: `studentId`
- Public constructor: `Student(String name, int age, String studentId)`
- Use `super(name, age)`.
- Public getter: `getStudentId()`

3. Complete `findOldest()`

- Accept an array of `Student` objects.
- Find and return the oldest student object.

Expected output:

```
Oldest Student: Carol
Age: 23
```

Exercise 2: Animal Sound System

Concepts: Method overriding, polymorphism, arrays of parent-class references

Given: `AnimalTest.java`

```
public class AnimalTest {
    public static void main(String[] args) {
        Animal[] animals = {
            new Dog("Buddy"),
            new Cat("Luna"),
            new Dog("Max")
        };

        for (Animal animal : animals) {
            System.out.print(animal.getName() + ": ");
            animal.makeSound();
        }
    }
}
```

Your tasks

1. `Animal.java`

- Private field: `name`
- Public constructor: `Animal(String name)`
- Public getter: `getName()`
- Public method: `makeSound()` prints `"Animal makes a sound"`

2. `Dog.java`

- Extend `Animal`.
- Public constructor: `Dog(String name)`
- Override `makeSound()` to print `"Woof!"`

3. `Cat.java`

- Extend `Animal`.
- Public constructor: `Cat(String name)`
- Override `makeSound()` to print `"Meow!"`

Expected output:

```
Buddy: Woof!
Luna: Meow!
Max: Woof!
```

Exercise 3: School and Student

Concepts: Aggregation, constructors, private fields, getters

Given: `SchoolTest.java`

```
public class SchoolTest {
    public static void main(String[] args) {
        School school = new School(
            "Central School",
            "New York"
        );

        Student[] students = {
            new Student("Alice", school),
            new Student("Bob", school),
            new Student("Carol", school)
        };

        for (Student student : students) {
            student.displayInfo();
            System.out.println();
        }
    }
}
```

Your tasks

1. `School.java`

- Private fields: `schoolName` and `address`
- Public constructor: `School(String schoolName, String address)`
- Public getters for both fields

2. `Student.java`

- Private fields: `name` and `school`
- The `school` field must have type `School`.
- Public constructor: `Student(String name, School school)`
- Public method: `displayInfo()` prints the student's name, school name, and address.
- Use getter methods to access the school's private fields.
- Do not extend `School`.

Expected output:

```
Name: Alice
School: Central School
Address: New York

Name: Bob
School: Central School
Address: New York

Name: Carol
School: Central School
Address: New York
```

Exercise 4: Pet Store Inventory

Concepts: Two-level inheritance, constructors, `super()`, private fields, arrays, searching

Given: `PetTest.java`

```
public class PetTest {
    public static void main(String[] args) {
        Puppy[] puppies = {
```

```
            new Puppy("Max", 500, "Beagle", 4),
            new Puppy("Bella", 700, "Poodle", 5),
            new Puppy("Coco", 450, "Pug", 3),
            new Puppy("Rocky", 600, "Husky", 6)
        };

        Puppy expensive = findMostExpensive(puppies);

        System.out.println("Most Expensive Puppy");
        System.out.println("Name: " + expensive.getName());
        System.out.println("Price: " + expensive.getPrice());
        System.out.println("Breed: " + expensive.getBreed());
        System.out.println(
            "Age: " + expensive.getAgeInMonths() + " months"
        );
    }

    public static Puppy findMostExpensive(Puppy[] puppies) {
        // TODO: Find and return the most expensive puppy.
        return null;
    }
}
```

Your tasks

1. `Pet.java`

- Private fields: `name` and `price`
- Public constructor: `Pet(String name, double price)`
- Public getters: `getName()` and `getPrice()`

2. `Dog.java`

- Extend `Pet`.
- Private field: `breed`
- Public constructor: `Dog(String name, double price, String breed)`
- Use `super(name, price).`
- Public getter: `getBreed()`

3. `Puppy.java`

- Extend `Dog`.
- Private field: `ageInMonths`
- Public constructor: `Puppy(String name, double price, String breed, int ageInMonths)`
- Use `super(name, price, breed).`
- Public getter: `getAgeInMonths()`

4. Complete `findMostExpensive()`

- Accept an array of `Puppy` objects.
- Find and return the puppy with the highest price.

Expected output:

```
Most Expensive Puppy
Name: Bella
Price: 700.0
Breed: Poodle
Age: 5 months
```

Requirements

1. Submit all completed `.java` files.
2. Do not change the provided `main()` methods.
3. Use `private` for all fields.
4. Use `public` for the required constructors and methods.
5. Use `extends` and `super()` where specified.
6. Do not use `protected`.
7. Run each test program and verify that the output matches the expected output.